COLLECTION GÉNIE LOGICIEL · LIVRE 1

Python

par la pratique

Partie 3 : organiser les données

Génie logiciel DQP / DTS

Option développement d'application

Moudériko Mouansié

Formateur référent en développement d'applications

MINEFOP Cameroun

Édition ASSIIN

Association des Informaticiens Innovants

Première version · Partie 3

Python par la pratique

Livre 1, Partie 3 : organiser les données (listes, tuples, dictionnaires, ensembles, compréhensions)

Auteur : **Moudériko Mouansié**
Formateur référent en développement d'applications, MINEFOP Cameroun

Édition : **ASSIIN** (Association des Informaticiens Innovants)

Public : étudiants en Génie logiciel DQP / DTS, option développement d'application

Version de référence : **Python 3.15**. Première version, contenant les chapitres 8 à 10, trois travaux pratiques, un lexique et une fiche mémo.

© Moudériko Mouansié, Édition ASSIIN. Tous droits réservés. Toute reproduction, même partielle, est soumise à l'autorisation de l'auteur.

Sommaire

Avant-propos de la Partie 3 4

Partie 3 : Organiser les données 5

Chapitre 8 : Les listes et les tuples 6

Chapitre 9 : Les dictionnaires et les ensembles 12

Chapitre 10 : Les compréhensions et les outils sur les collections 17

Travaux pratiques de la Partie 3 20

TP 7 : Le bilan de notes d'une classe 21

TP 8 : Un analyseur de texte 23

TP 9 : Un carnet d'adresses 24

Lexique et fiche mémo de la Partie 3 26

Avant-propos de la Partie 3

Ce volume est la **troisième partie** du Livre 1, *Python par la pratique*, destiné aux étudiants de Génie logiciel (DQP et DTS), option développement d'application. Il se lit seul, à condition d'avoir assimilé les Parties 1 et 2.

Ce que vous devez déjà savoir

- utiliser l'invite de commandes, l'interpréteur Python et les scripts ;
- manipuler variables, types, opérateurs, `input()`, `print()` et f-strings ;
- écrire des conditions, des boucles `while` et `for`, et manipuler des chaînes.

Ce que vous allez apprendre

Jusqu'ici, vous manipuliez surtout des valeurs isolées. Dans cette partie, vous apprenez à **organiser** des ensembles de données : les **listes** et les **tuples** (chapitre 8), les **dictionnaires** et les **ensembles** (chapitre 9), puis les **compréhensions** (chapitre 10), qui permettent de transformer ces données en une ligne.

La méthode de travail est inchangée : lisez, reprenez chaque exemple dans l'interpréteur, expérimentez, puis faites les exercices (**non corrigés**, avec « Pour vérifier » quand c'est utile) et les trois TP de fin de partie.

> **Cette version**
> Ce volume contient les chapitres 8 à 10, trois travaux pratiques, un lexique et une fiche mémo. Plan du Livre 1 : Partie 1 (chapitres 1 à 4), Partie 2 (5 à 7), **Partie 3 (8 à 10)**, puis les parties suivantes.

Partie 3
Organiser les données

Chapitre 8 : Les listes et les tuples

Objectifs du chapitre

- créer une liste, accéder à ses éléments et la modifier ;
- utiliser les principales méthodes de listes et les fonctions `len`, `sum`, `min`, `max`, `sorted` ;
- parcourir une liste avec `for`, `enumerate` et `zip` ;
- éviter le piège de la copie de liste ;
- utiliser les tuples et le dépaquetage.

8.1 Ranger plusieurs valeurs dans une liste

Une variable ne contient qu'une valeur. Pour mémoriser les notes de toute une classe, il serait absurde de créer une variable par note. Une **liste** range plusieurs valeurs, dans l'ordre, sous un seul nom. Elle s'écrit entre crochets, valeurs séparées par des virgules. Comme pour les chaînes, on accède aux éléments par leur **indice** (à partir de 0) et on extrait des morceaux avec des **tranches**.

Dans l'interpréteur Python

```
>>> notes = [12, 15.5, 9, 18]
>>> notes
[12, 15.5, 9, 18]
>>> len(notes)
4
>>> notes[0]
12
>>> notes[-1]
18
>>> notes[1:3]
[15.5, 9]
>>> melange = ["Awa", 20, True]
>>> melange
['Awa', 20, True]
>>> vide = []
>>> list("abc")
['a', 'b', 'c']
>>> list(range(4))
[0, 1, 2, 3]
```

Un indice hors limites

```
>>> notes[10]
Traceback (most recent call last):
  File "<python-input-12>", line 1, in <module>
    notes[10]
IndexError: list index out of range
```

8.2 Modifier une liste

Contrairement aux chaînes, les listes sont **modifiables** : on peut changer, ajouter ou retirer des éléments sur place.

Dans l'interpréteur Python

```
>>> L = [12, 15, 9, 18]
>>> L[0] = 14
>>> L
[14, 15, 9, 18]
>>> L.append(10)
>>> L
[14, 15, 9, 18, 10]
>>> L.insert(1, 11)
>>> L
[14, 11, 15, 9, 18, 10]
>>> L.remove(9)
>>> L
[14, 11, 15, 18, 10]
>>> dernier = L.pop()
>>> dernier
10
>>> L
[14, 11, 15, 18]
>>> del L[0]
>>> L
[11, 15, 18]
```

Méthode	Effet
`append(x)`	Ajoute `x` à la fin.
`insert(i, x)`	Insère `x` à la position `i`.
`extend(autre)`	Ajoute tous les éléments d'une autre liste.
`remove(x)`	Supprime la première occurrence de `x`.
`pop()` / `pop(i)`	Retire et renvoie le dernier élément (ou celui d'indice `i`).
`sort()` / `reverse()`	Trie / inverse la liste sur place.
`index(x)` / `count(x)`	Position de la première occurrence / nombre d'occurrences.

Plusieurs fonctions s'appliquent à une liste de nombres. Attention à la différence entre `sorted(L)` (renvoie une **nouvelle** liste triée) et `L.sort()` (trie la liste **elle-même**).

Dans l'interpréteur Python

```
>>> L = [12, 15, 9, 18, 10]
>>> sorted(L)
[9, 10, 12, 15, 18]
>>> L
[12, 15, 9, 18, 10]
>>> L.sort(reverse=True)
>>> L
[18, 15, 12, 10, 9]
>>> min(L)
9
>>> max(L)
18
>>> sum(L)
64
>>> sum(L) / len(L)
12.8
>>> 15 in L
True
>>> L.index(9)
4
```

Le piège de la copie

Écrire `b = a` ne copie pas la liste : les deux noms désignent **la même** liste. Pour obtenir une copie indépendante, utilisez `a.copy()` (ou `a[:]`).

Dans l'interpréteur Python

```
>>> a = [1, 2, 3]
>>> b = a
>>> b.append(4)
>>> a
[1, 2, 3, 4]
>>> c = a.copy()
>>> c.append(5)
>>> a
[1, 2, 3, 4]
>>> c
[1, 2, 3, 4, 5]
```

Points d'attention

• Pour les nombres et les textes, `b = a` crée une copie indépendante (chapitre 3). Pour les listes, ce n'est pas le cas : la liste est un objet **modifiable**, et les deux noms pointent vers le même objet.

8.3 Parcourir une liste

La boucle `for` parcourt directement les éléments. Pour avoir aussi leur numéro, utilisez `enumerate()` ; pour parcourir deux listes en parallèle, `zip()`.

Dans l'interpréteur Python

```
>>> fruits = ["mangue", "ananas", "papaye"]
>>> for f in fruits:
...     print(f)
...
mangue
ananas
papaye
>>> for i, f in enumerate(fruits, start=1):
...     print(i, f)
...
1 mangue
2 ananas
3 papaye
>>> noms = ["Awa", "Brice"]
>>> notes = [14, 10]
>>> for nom, note in zip(noms, notes):
...     print(nom, note)
...
Awa 14
Brice 10
```

8.4 Les listes de listes

Les éléments d'une liste peuvent eux-mêmes être des listes : c'est ainsi qu'on représente un tableau à deux dimensions. On accède à un élément avec deux indices : `grille[ligne][colonne]`.

Dans l'interpréteur Python

```
>>> grille = [[1, 2, 3], [4, 5, 6]]
>>> grille[1]
[4, 5, 6]
>>> grille[1][2]
6
>>> len(grille)
2
>>> for ligne in grille:
...     print(ligne)
...
[1, 2, 3]
[4, 5, 6]
```

8.5 Passer du texte aux listes

Les méthodes `split()` et `join()` du chapitre 7 prennent tout leur sens :

Dans l'interpréteur Python

```
>>> mots = "le chat dort".split()
>>> mots
['le', 'chat', 'dort']
>>> mots[1]
'chat'
>>> " ".join(mots)
'le chat dort'
>>> "-".join(mots)
'le-chat-dort'
```

8.6 Les tuples

Un **tuple** ressemble à une liste, mais il est **immuable** : on ne peut ni changer, ni ajouter, ni supprimer un élément. Il s'écrit entre parenthèses. On l'utilise pour des valeurs qui vont ensemble et ne doivent pas changer (des coordonnées, une date...). Un tuple à un seul élément s'écrit avec une virgule : `(5,)`.

Dans l'interpréteur Python

```
>>> point = (3, 4)
>>> point[0]
3
>>> x, y = point
>>> x
3
>>> y
4
>>> seul = (5,)
>>> type(seul)
<class 'tuple'>
>>> list(point)
[3, 4]
>>> tuple([1, 2])
(1, 2)
```

Un tuple est immuable

```
>>> point[0] = 9
Traceback (most recent call last):
  File "<python-input-30>", line 1, in <module>
    point[0] = 9
TypeError: 'tuple' object does not support item assignment
```

L'écriture `x, y = point` s'appelle le **dépaquetage** : elle répartit les éléments dans plusieurs variables. C'est ce qui permettait l'échange `a, b = b, a` vu au chapitre 3.

À retenir

- Une liste `[a, b, c]` est ordonnée et modifiable ; ses éléments s'atteignent par indice et par tranche.
- `append`, `insert`, `remove`, `pop`, `sort` modifient la liste sur place ; `sorted` renvoie une nouvelle liste.
- `b = a` ne copie pas une liste : utilisez `a.copy()`.
- `enumerate()` donne indice et valeur ; `zip()` parcourt plusieurs listes en parallèle.
- Un tuple `(a, b)` est une liste immuable ; le dépaquetage `x, y = point` répartit ses éléments.

Exercices

Exercice 8.1 ★☆☆

Créez une liste de cinq notes. Affichez la première, la dernière et les trois premières.

Exercice 8.2 ★☆☆

Avec la liste `[12, 15, 9, 18, 10]`, calculez la moyenne, la plus petite et la plus grande note.

Pour vérifier : 12.8, 9 et 18.

Exercice 8.3 ★☆☆
Partez de `[12, 15, 9, 18, 10]` : ajoutez la note 14, retirez la note 9, puis triez la liste par ordre décroissant.

Pour vérifier : [18, 15, 14, 12, 10].

Exercice 8.4 ★★☆
Demandez des entiers jusqu'à ce que l'utilisateur tape 0. Affichez la liste triée et la somme.

Pour vérifier : 5, 3, 8, 0 donne [3, 5, 8] et 16.

Exercice 8.5 ★★☆
Supprimez les doublons d'une liste en conservant l'ordre d'apparition.

Pour vérifier : [3, 1, 3, 2, 1] donne [3, 1, 2].

Exercice 8.6 ★★☆
Séparez les entiers de 1 à 10 en deux listes : les pairs et les impairs.

Pour vérifier : [2, 4, 6, 8, 10] et [1, 3, 5, 7, 9].

Exercice 8.7 ★★☆
Avec `[[1, 2, 3], [4, 5, 6]]`, affichez la somme de chaque ligne puis de chaque colonne.

Pour vérifier : lignes : 6 et 15 ; colonnes : 5, 7 et 9.

Exercice 8.8 ★★★
Trouvez le deuxième plus grand élément d'une liste, sans utiliser `sort()` ni `sorted()`.

Pour vérifier : pour [12, 15, 9, 18, 10], le résultat est 15.

Exercice 8.9 ★★☆
Prédisez le contenu de `a` et de `b` après : `a = [1, 2]`, `b = a`, `b.append(3)`. Vérifiez, puis corrigez le code pour que `a` reste inchangée.

Testez-vous

1. Quelle différence entre une liste et un tuple ?
2. Quelle différence entre `L.sort()` et `sorted(L)` ?
3. Pourquoi modifier `b` modifie-t-il aussi `a` après `b = a` ?
4. À quoi sert `enumerate()` ?

Chapitre 9 : Les dictionnaires et les ensembles

Objectifs du chapitre

- associer des valeurs à des clés avec un dictionnaire ;
- ajouter, lire, modifier, supprimer et parcourir des entrées ;
- utiliser un ensemble pour éliminer les doublons et comparer des collections ;
- choisir la structure de données adaptée.

9.1 Le dictionnaire : des valeurs repérées par des clés

Dans une liste, on retrouve un élément par sa position. Dans un **dictionnaire**, on le retrouve par un **nom**, appelé **clé**. Il s'écrit entre accolades, sous la forme `clé: valeur`, et décrit très bien un objet réel comme la fiche d'un étudiant.

Dans l'interpréteur Python

```
>>> etudiant = {"nom": "Mballa", "prenom": "Jean", "age": 20}
>>> etudiant["nom"]
'Mballa'
>>> etudiant["age"] = 21
>>> etudiant["ville"] = "Douala"
>>> etudiant
{'nom': 'Mballa', 'prenom': 'Jean', 'age': 21, 'ville': 'Douala'}
>>> etudiant.get("email")
>>> etudiant.get("email", "inconnu")
'inconnu'
>>> "nom" in etudiant
True
>>> len(etudiant)
4
```

Une clé absente

```
>>> etudiant["email"]
Traceback (most recent call last):
  File "<python-input-8>", line 1, in <module>
    etudiant["email"]
KeyError: 'email'
```

Lire une clé qui n'existe pas provoque une `KeyError`. La méthode `get(clé, défaut)` évite l'erreur en renvoyant une valeur par défaut (`None` si on n'en donne pas). Les clés doivent être **uniques** et **immuables** (texte, nombre, tuple). Depuis Python 3.7, un dictionnaire conserve l'**ordre d'insertion**.

Parcourir et supprimer

Dans l'interpréteur Python

```
>>> etudiant = {"nom": "Mballa", "prenom": "Jean", "age": 20, "ville": "Douala"}
>>> etudiant.keys()
dict_keys(['nom', 'prenom', 'age', 'ville'])
>>> etudiant.values()
dict_values(['Mballa', 'Jean', 20, 'Douala'])
>>> etudiant.items()
dict_items([('nom', 'Mballa'), ('prenom', 'Jean'), ('age', 20), ('ville', 'Douala')])
>>> for cle, valeur in etudiant.items():
...     print(cle, ":", valeur)
...
nom : Mballa
prenom : Jean
age : 20
ville : Douala
>>> etudiant.pop("ville")
'Douala'
>>> del etudiant["age"]
>>> etudiant
{'nom': 'Mballa', 'prenom': 'Jean'}
```

Un motif très utile : compter

Dans l'interpréteur Python

```
>>> compte = {}
>>> for lettre in "banane":
...     compte[lettre] = compte.get(lettre, 0) + 1
...
>>> compte
{'b': 1, 'a': 2, 'n': 2, 'e': 1}
```

Structures imbriquées

Valeurs et clés peuvent être des listes ou des dictionnaires : une **liste de dictionnaires** représente un tableau de fiches, un **dictionnaire de listes** regroupe des séries de valeurs.

Dans l'interpréteur Python

```
>>> classe = [{"nom": "Awa", "note": 14}, {"nom": "Brice", "note": 10}]
>>> for e in classe:
...     print(e["nom"], e["note"])
...
Awa 14
Brice 10
>>> classe[0]["note"]
14
>>> notes = {"Awa": [14, 16], "Brice": [10, 9]}
>>> sum(notes["Awa"]) / len(notes["Awa"])
15.0
```

Nouveauté de Python 3.15

Python 3.15 ajoute un type intégré **frozendict**, un dictionnaire non modifiable, équivalent pour les dictionnaires de ce que le tuple est pour les listes. Il est réservé à des usages avancés : retenez simplement qu'il existe.

9.2 Les ensembles

Un **ensemble** (*set*) est une collection de valeurs **uniques** et **sans ordre**. Il s'écrit entre accolades, mais sans clés. Il est idéal pour éliminer les doublons et pour les opérations de la théorie des ensembles.

Dans l'interpréteur Python

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5}
>>> a | b
{1, 2, 3, 4, 5}
>>> a & b
{3, 4}
>>> a - b
{1, 2}
>>> a ^ b
{1, 2, 5}
>>> a.add(9)
>>> a
{1, 2, 3, 4, 9}
>>> len(a)
5
>>> 3 in a
True
>>> sorted(set([3, 1, 3, 2, 1]))
[1, 2, 3]
>>> type({})
<class 'dict'>
>>> vide = set()
>>> type(vide)
<class 'set'>
```

Opération	Symbole	Résultat
Union	`a | b`	Éléments de a ou de b.
Intersection	`a & b`	Éléments communs à a et à b.
Différence	`a - b`	Éléments de a absents de b.
Différence symétrique	`a ^ b`	Éléments présents dans un seul des deux.

Points d'attention

- `{}` désigne un **dictionnaire vide**, pas un ensemble vide : pour un ensemble vide, écrivez `set()`.
- L'ordre d'affichage d'un ensemble n'est pas garanti : ne comptez jamais dessus.

9.3 Choisir la bonne structure

Structure	Ordonnée	Modifiable	Doublons	Utile pour…
Liste	oui	oui	oui	Une série d'éléments qui évolue.
Tuple	oui	non	oui	Un petit groupe de valeurs fixes.
Dictionnaire	oui (insertion)	oui	clés uniques	Retrouver une valeur par un nom.
Ensemble	non	oui	non	Unicité, comparaisons d'ensembles.

À retenir

- Un dictionnaire associe des clés uniques à des valeurs : `d[clé]`, `d.get(clé, défaut)`, `d.items()`.
- Lire une clé absente avec `d[clé]` provoque une KeyError ; `get` l'évite.
- Compter : `d[x] = d.get(x, 0) + 1`.
- Un ensemble ne contient que des valeurs uniques, sans ordre ; `|`, `&`, `-`, `^` combinent des ensembles.
- `{}` est un dictionnaire vide ; `set()` est un ensemble vide.

Exercices

Exercice 9.1 ★☆☆

Créez le dictionnaire d'une personne (nom, âge, ville) et affichez chaque paire clé-valeur avec `items()`.

Exercice 9.2 ★☆☆

Partant de ce dictionnaire, ajoutez une clé, modifiez l'âge, puis supprimez la ville.

Exercice 9.3 ★★☆

Comptez les occurrences de chaque lettre d'un mot.

Pour vérifier : « ananas » donne a : 3, n : 2, s : 1.

Exercice 9.4 ★★☆

Comptez les occurrences de chaque mot d'une phrase.

Pour vérifier : « le chat et le chien et le rat » donne le : 3, et : 2, chat : 1, chien : 1, rat : 1.

Exercice 9.5 ★★☆

Inversez un dictionnaire (les valeurs deviennent les clés, et inversement).

Pour vérifier : {"a": 1, "b": 2} donne {1: "a", 2: "b"}.

Exercice 9.6 ★★☆

À partir de deux listes, `noms` et `telephones`, construisez un dictionnaire nom vers téléphone (aidez-vous de `zip()`).

Exercice 9.7 ★★☆

Avec deux ensembles d'inscrits, `{"Awa", "Brice", "Carine"}` en Python et `{"Brice", "Didier"}` en algorithmique, trouvez ceux inscrits aux deux, ceux inscrits à au moins un cours, et ceux inscrits en Python uniquement.

Pour vérifier : Brice ; 4 étudiants ; Awa et Carine (l'ordre d'affichage d'un ensemble n'est pas garanti).

Exercice 9.8 ★★★

À partir de `{"Awa": [14, 16, 12], "Brice": [9, 11, 10]}`, affichez la moyenne de chaque étudiant.

Pour vérifier : Awa : 14.0 ; Brice : 10.0.

Exercice 9.9 ★★★

Deux mots sont des anagrammes s'ils contiennent exactement les mêmes lettres. Écrivez un test (aidez-vous de `sorted()`).

Pour vérifier : « chien » et « niche » : oui ; « chat » et « chats » : non.

Testez-vous

1. Quelle différence entre `d[cle]` et `d.get(cle)` ?
2. Pourquoi `{}` ne crée-t-il pas un ensemble vide ?
3. Dans quels cas préférer un ensemble à une liste ?
4. Citez une structure qui peut servir de clé de dictionnaire et une qui ne le peut pas.

Chapitre 10 : Les compréhensions et les outils sur les collections

Objectifs du chapitre

- construire une liste, un dictionnaire ou un ensemble en une ligne ;
- filtrer et transformer des données ;
- utiliser `sorted`, `sum`, `any`, `all`.

10.1 La compréhension de liste

Construire une liste avec une boucle est si courant que Python offre une écriture plus courte : la **compréhension de liste**. Comparez :

Deux écritures du même calcul

```
# Avec une boucle                    # Avec une compréhension
carres = []
for n in range(1, 6):                carres = [n ** 2 for n in range(1, 6)]
    carres.append(n ** 2)
```

La forme générale est `[expression for variable in série if condition]`. La partie `if` est facultative : elle **filtre** les éléments.

Dans l'interpréteur Python

```
>>> [n ** 2 for n in range(1, 6)]
[1, 4, 9, 16, 25]
>>> [n for n in range(1, 11) if n % 2 == 0]
[2, 4, 6, 8, 10]
>>> mots = ["python", "java", "c"]
>>> [m.upper() for m in mots]
['PYTHON', 'JAVA', 'C']
>>> [len(m) for m in mots]
[6, 4, 1]
>>> ["long" if len(m) > 3 else "court" for m in mots]
['long', 'long', 'court']
>>> [(x, y) for x in range(2) for y in range(2)]
[(0, 0), (0, 1), (1, 0), (1, 1)]
```

Ne confondez pas les deux places du `if` : à la **fin**, il filtre ; au **début** (avec un `else`), il choisit la valeur de chaque élément.

10.2 Dictionnaires et ensembles en compréhension

Dans l'interpréteur Python

```
>>> {n: n ** 2 for n in range(1, 5)}
{1: 1, 2: 4, 3: 9, 4: 16}
>>> mots = ["python", "java", "c"]
>>> {m: len(m) for m in mots}
{'python': 6, 'java': 4, 'c': 1}
>>> {len(m) for m in mots}
{1, 4, 6}
```

10.3 Des fonctions pratiques

Dans l'interpréteur Python

```
>>> notes = [14, 9, 17, 11]
>>> sorted(notes)
[9, 11, 14, 17]
>>> sorted(notes, reverse=True)
[17, 14, 11, 9]
>>> mots = ["python", "c", "java"]
>>> sorted(mots, key=len)
['c', 'java', 'python']
>>> any(n < 10 for n in notes)
True
>>> all(n >= 10 for n in notes)
False
>>> sum(n for n in notes if n >= 10)
42
>>> list(zip(["a", "b"], [1, 2]))
[('a', 1), ('b', 2)]
```

Fonction	Rôle
`sorted(x, key=f, reverse=True)`	Liste triée ; `key` indique le critère, `reverse` l'ordre décroissant.
`sum(x)`, `min(x)`, `max(x)`	Somme, plus petit, plus grand élément.
`any(x)` / `all(x)`	Au moins un élément vrai / tous les éléments vrais.
`zip(a, b)`, `enumerate(x)`	Parcours en parallèle, avec numéro d'ordre.

Points d'attention

• Une compréhension trop longue devient illisible. Si elle ne tient pas sur une ligne claire, écrivez une vraie boucle : la lisibilité passe avant la concision.

À retenir

• `[expression for x in série if condition]` construit une liste ; avec des accolades, un dictionnaire ou un ensemble.

• Le `if` final filtre ; l'expression conditionnelle placée au début transforme.

• `sorted`, `sum`, `min`, `max`, `any`, `all` s'appliquent à toute collection.

• Privilégiez la lisibilité : une compréhension doit rester courte.

Exercices

Exercice 10.1 ★☆☆

Construisez par compréhension la liste des cubes des entiers de 1 à 10.

Pour vérifier : le dernier élément est 1000.

Exercice 10.2 ★☆☆

Construisez la liste des multiples de 3 jusqu'à 30.

Pour vérifier : [3, 6, 9, 12, 15, 18, 21, 24, 27, 30].

Exercice 10.3 ★☆☆
À partir d'une liste de mots, construisez la liste de leurs longueurs.

Exercice 10.4 ★★☆
Mettez en majuscule la première lettre de chaque mot d'une liste.

Pour vérifier : ["awa", "brice"] donne ["Awa", "Brice"].

Exercice 10.5 ★★☆
Parmi `[14, 9, 17, 11, 6]`, ne gardez que les notes d'au moins 10.

Pour vérifier : [14, 17, 11].

Exercice 10.6 ★★☆
Construisez le dictionnaire des carrés des entiers de 1 à 5.

Pour vérifier : {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}.

Exercice 10.7 ★★★
Construisez la liste des diviseurs d'un entier saisi. Utilisez-la pour dire si un nombre est parfait (égal à la somme de ses diviseurs autres que lui-même).

Pour vérifier : 12 a pour diviseurs [1, 2, 3, 4, 6, 12] ; 28 est parfait.

Exercice 10.8 ★★★
Construisez, en une seule compréhension, la liste des nombres premiers jusqu'à 50 (aidez-vous de `all()`).

Pour vérifier : [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47].

> **Testez-vous**
> 1. Que fait chaque partie de `[n ** 2 for n in range(5) if n % 2 == 0]` ?
> 2. Quelle différence entre le `if` placé à la fin et celui placé au début ?
> 3. Comment trier des mots par longueur ?
> 4. Que renvoient `any([])` et `all([])` ?

Travaux pratiques de la Partie 3

Ces trois TP mobilisent les listes, les dictionnaires, les ensembles et les compréhensions, avec les notions des parties précédentes. Même méthode : un script par TP, lancé depuis l'invite de commandes, étape par étape, en comparant votre affichage à l'exemple.

TP 7 : Le bilan de notes d'une classe

Compétences mobilisées

Dictionnaire de listes, compréhension de dictionnaire, `sorted()` avec `key`, `enumerate()`, alignement avec les f-strings.

Durée estimée : 2 h.

Énoncé

Écrivez un script **bilan.py** qui analyse les notes d'une classe. Les données sont fournies dans le script, sous la forme d'un dictionnaire de listes :

Les données

```
notes = {
    "Awa": [14, 16, 12],
    "Brice": [9, 11, 10],
    "Carine": [17, 15, 18],
    "Didier": [8, 12, 7],
}
```

Étape 1 : les moyennes

Construisez, par compréhension de dictionnaire, un dictionnaire des moyennes par étudiant.

Étape 2 : le tableau

Affichez un tableau avec le nom (aligné à gauche sur 10), la liste des notes (sur 15), la moyenne (sur 8, deux décimales) et la mention (« Très bien » à partir de 16, « Bien » à partir de 14, « Assez bien » à partir de 12, « Passable » à partir de 10, sinon « Ajourné »).

Étape 3 : le classement et les statistiques

Affichez le classement par moyenne décroissante (avec `sorted(moyennes, key=moyennes.get, reverse=True)`), la moyenne de la classe, le meilleur et le plus faible résultat.

Exemple d'exécution

```
Étudiant  Notes           Moyenne  Mention
Awa       [14, 16, 12]      14.00  Bien
Brice     [9, 11, 10]       10.00  Passable
Carine    [17, 15, 18]      16.67  Très bien
Didier    [8, 12, 7]         9.00  Ajourné

Classement :
1. Carine     16.67
2. Awa        14.00
3. Brice      10.00
4. Didier      9.00

Moyenne de la classe : 12.42 / 20
Meilleur résultat : Carine (16.67)
Résultat le plus faible : Didier (9.00)
```

Pour aller plus loin (facultatif)

- Ajoutez la note maximale de chaque étudiant.
- Affichez le nombre d’étudiants admis (moyenne d’au moins 10).
- Après la Partie 4, regroupez les calculs dans des fonctions.

Critères de réussite

- Votre affichage est identique à l’exemple.
- Ajouter un étudiant dans le dictionnaire met le bilan à jour sans autre modification.
- Aucun calcul n’est répété : les moyennes sont calculées une seule fois.

TP 8 : Un analyseur de texte

Compétences mobilisées

Méthodes de chaînes, listes, ensembles, dictionnaire de comptage, tri avec `key`.

Durée estimée : 2 h.

Énoncé

Écrivez un script **analyse.py** qui analyse le texte suivant :

Le texte à analyser

```
texte = ("Python est un langage simple. Python est puissant, et Python est partout. "
         "Apprendre Python, c'est facile et utile !")
```

Étape 1 : nettoyer et découper

Mettez le texte en minuscules, remplacez les signes de ponctuation (`. , ; ! ?`) par des espaces avec `replace()`, puis découpez-le en une liste de mots avec `split()`.

Étape 2 : compter

Affichez le nombre de mots, le nombre de mots **différents** (aidez-vous d'un ensemble) et le mot le plus long.

Étape 3 : les mots les plus fréquents

Construisez un dictionnaire de comptage (motif vu au chapitre 9), puis affichez les trois mots les plus fréquents avec leur nombre d'occurrences, alignés comme dans l'exemple.

Exemple d'exécution

```
Nombre de mots : 18
Mots différents : 12
Mot le plus long : apprendre
Les 3 mots les plus fréquents :
  python  4
  est     3
  et      2
```

Pour aller plus loin (facultatif)

- Affichez aussi la longueur moyenne des mots.
- Demandez à l'utilisateur de saisir son propre texte.
- Affichez les mots qui n'apparaissent qu'une seule fois.

Critères de réussite

- Votre affichage est identique à l'exemple.
- Le programme fonctionne pour n'importe quel texte, pas seulement celui de l'énoncé.
- Le nettoyage de la ponctuation est fait dans une boucle, sans répétition de code.

TP 9 : Un carnet d'adresses

> **Compétences mobilisées**
> Dictionnaire, boucle `while True` avec `break`, `if / elif / else`, `in`, `del`, `sorted()`.
> Durée estimée : 2 h 30.

Énoncé

Écrivez un script **carnet.py** qui gère un carnet d'adresses stocké dans un **dictionnaire** (nom vers numéro de téléphone). Le programme affiche un menu et recommence jusqu'à ce que l'utilisateur choisisse de quitter.

Étape 1 : le menu

Affichez le menu ci-dessous, lisez le choix de l'utilisateur dans une boucle `while True`, et traitez le choix « 0 » (message « Au revoir ! » puis sortie de la boucle) et les choix invalides.

Étape 2 : ajouter et rechercher

Le choix 1 demande un nom et un téléphone puis enregistre le contact. Le choix 2 demande un nom et affiche le téléphone, ou « ... est introuvable. » si le nom est absent (testez avec `in`).

Étape 3 : lister et supprimer

Le choix 3 affiche tous les contacts, **triés par nom**, avec le nom aligné sur 10 caractères (ou « Le carnet est vide. »). Le choix 4 supprime un contact avec `del`, en gérant le cas où il n'existe pas.

Exemple d'exécution

```
[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 1
Nom : Awa
Téléphone : 677000001
Awa ajouté.

[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 1
Nom : Brice
Téléphone : 699000002
Brice ajouté.

[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 3
  Awa       677000001
  Brice     699000002

[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 2
Nom : Zoe
Zoe est introuvable.

[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 4
Nom : Brice
Brice supprimé.

[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 3
  Awa       677000001

[1] Ajouter  [2] Rechercher  [3] Lister  [4] Supprimer  [0] Quitter
Votre choix : 0
Au revoir !
```

Pour aller plus loin (facultatif)

- Refusez d’ajouter un nom déjà présent, ou proposez de le mettre à jour.
- Permettez de rechercher par début de nom.
- Après le chapitre sur les fichiers, sauvegardez le carnet entre deux exécutions.

Critères de réussite

- Votre programme reproduit l’exemple pour les mêmes saisies.
- Un nom inconnu ne fait jamais planter le programme.
- Le menu est affiché une seule fois dans le code, mais à chaque tour d’exécution.

Lexique et fiche mémo de la Partie 3

Lexique

Terme	Définition
Liste	Collection ordonnée et modifiable d'éléments, notée entre crochets.
Tuple	Collection ordonnée et immuable d'éléments, notée entre parenthèses.
Dictionnaire	Collection de paires clé-valeur, notée entre accolades.
Clé	Nom qui permet de retrouver une valeur dans un dictionnaire (unique, immuable).
Ensemble (set)	Collection de valeurs uniques et sans ordre.
Élément	Une valeur contenue dans une collection.
Dépaquetage	Répartition des éléments d'une collection dans plusieurs variables.
Modifiable / immuable	Qui peut / ne peut pas être changé sur place après sa création.
Alias	Deux noms qui désignent le même objet (cas de b = a avec une liste).
Compréhension	Écriture courte pour construire une liste, un dictionnaire ou un ensemble.
Itérable	Tout objet que l'on peut parcourir avec une boucle for.
KeyError	Erreur levée quand on lit une clé absente d'un dictionnaire.

Fiche mémo

Les structures de la Partie 3

```
# Liste                          # Dictionnaire
L = [1, 2, 3]                    d = {"nom": "Awa", "age": 20}
L.append(4)  L.remove(2)         d["ville"] = "Douala"
L.sort()     L.pop()             d.get("email", "inconnu")
L[0]  L[-1]  L[1:3]              for cle, val in d.items(): ...
for i, x in enumerate(L): ...    "nom" in d    del d["age"]

# Tuple et ensemble              # Compréhensions
p = (3, 4)   x, y = p            [n ** 2 for n in range(5) if n % 2 == 0]
s = {1, 2}   s | t  s & t  s - t {n: n ** 2 for n in range(5)}
set()  # ensemble vide           sorted(L, key=len, reverse=True)
```